

B1B13PPS

Průmyslové počítačové systémy

Michal Brejcha

`brejcmic@fel.cvut.cz`

ČVUT v Praze, FEL

Praha, 2025

Obsah

- 1 Popis protokolu
- 2 Ukázka komunikace
- 3 Tématické okruhy ke zkoušce

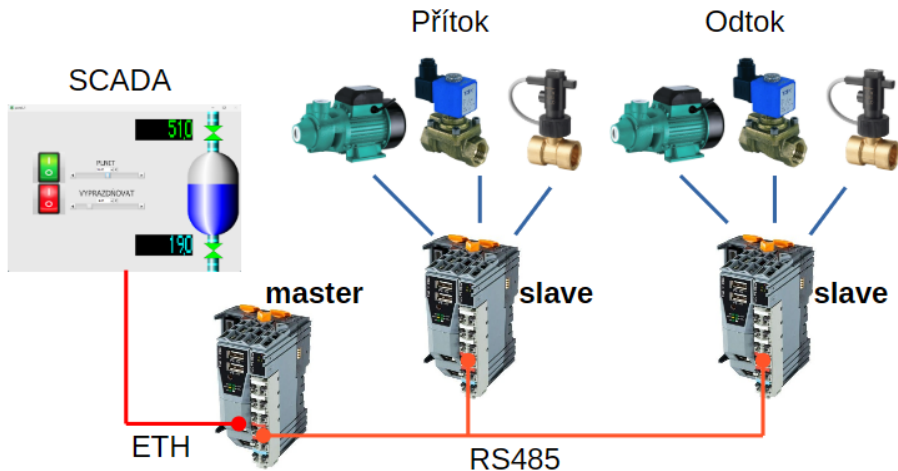
Téma

- 1 Popis protokolu
- 2 Ukázka komunikace
- 3 Tématické okruhy ke zkoušce

Historie a použití

- Protokol vytvořila firma MODICON v roce 1979,
- byl určen pro DCS (**D**istributed **C**ontrol **S**ystem), tj. více spolupracujících stanic (PLC) se podílí na řízení technologického procesu:
 - jednotlivé části technologie mají vlastní řídicí jednotku,
 - systémy spolu komunikují po průmyslové síti,
- primárně byl určen pro sériovou komunikaci - Modbus RTU,
- v roce 1999 přibyla podpora ethernetové komunikace - Modbus TCP,
- komunikace probíhá v režimu master-slave:
 - **master** zadává dotazy nebo příkazy,
 - **slave** odpovídá nebo provádí akce.

Typický DCS



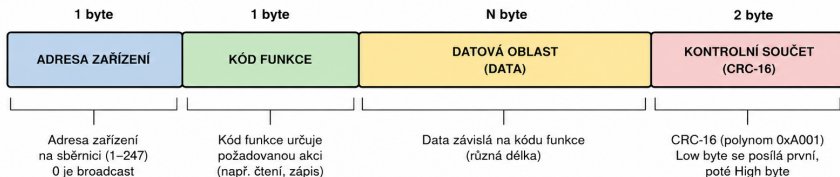
Datový model

- Protokol je založen na poskytování dat ve formě tabulek (nebo lépe paměťového prostoru),
- typický má paměťový prostor 4 oblasti (bloky paměti):

Adresa	Typ hodnoty	Vztah k PLC
00001 - 09999	Cívky	odpovídají digitální výstupy
10000 - 19999	Diskrétní vstupy	digitální vstupy
30000 - 39999	Vstupní registry	analogové vstupy
40000 - 49999	Uchovávací registry	analogové výstupy

- analogové hodnoty mají vždy rozlišení 16 bitů,
- diskrétní hodnoty jsou jednobitové.
- **POZOR:** je rozdíl mezi fyzickou a logickou adresací:
 - při **logické adresaci** má první buňka adresu **1**, zatímco
 - při **fyzické adresaci** má první buňka adresu **0**.

Formát zprávy



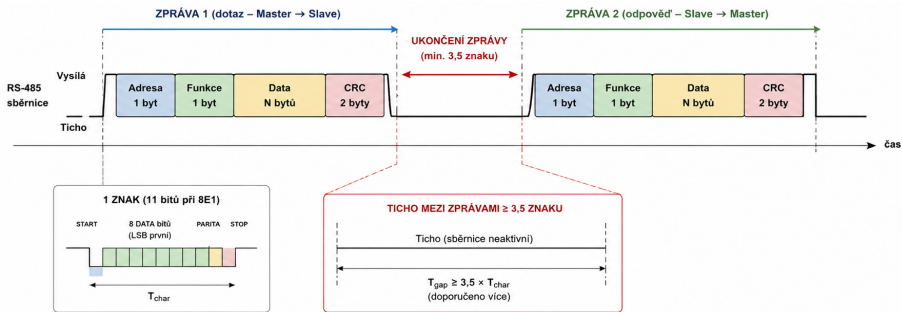
Příklad: Čtení 4 holding registrů od adresy 0x0000
(Function Code 03) od zařízení s adresou 0x01



Zpráva odesílaná masterem

- Data mají reprezentaci Big Endian,
- CRC má reprezentaci Little Endian,

Oddělení jednotlivých zpráv



- Klid na lince 3.5 znaku odděluje jednotlivé zprávy nebo ruší předchozí vysílání,

POZNÁMKA: Pro Modbus RTU je velmi typické nastavení sériové linky se sudou paritou.

Nejběžnější kódy funkcí

Kód	Význam
01	Read Coils – čtení stavů binárních výstupů (coil)
02	Read Discrete Inputs – čtení binárních vstupů pouze pro čtení
03	Read Holding Registers – čtení holding registrů
04	Read Input Registers – čtení vstupních registrů pouze pro čtení
05	Write Single Coil – zápis jednoho binárního výstupu
06	Write Single Register – zápis jednoho holding registru
08	Diagnostics – diagnostické funkce sériové linky
15	Write Multiple Coils – zápis více binárních výstupů
16	Write Multiple Registers – zápis více holding registrů
22	Mask Write Register – maskovaný zápis do registru
23	Read/Write Multiple Registers – současné čtení a zápis více registrů
43/14	Encapsulated Interface Transport – identifikace zařízení (Read Device Identification)

Adresa zařízení a CRC

Adresa zařízení:

- Číslo v rozmezí 1-247 (0 je broadcast, 248 až 255 jsou rezervované),
- musí být unikátní pro konkrétní stanici.

Cyclic Redundancy Check:

- metoda pro detekci chyb přenosu dat,
- modbus používá CRC-16,
- počítá se ze všech bytů zprávy při vysílání i příjmu,
- přijímač porovnává CRC ve zprávě s tím, které bylo vypočteno.

Příklad posílání jednotlivých bitů

1) ZPRÁVA: ČTI CÍVKY (Read Coils) – požadavek od Master → Slave

Adresa zařízení	Kód funkce	Počáteční adresa		Počet cívek		CRC (CRC-16)	
		Hi byte	Lo byte	Hi byte	Lo byte	Low byte	High byte
0x01 1 byte	0x01 1 byte	0x00 1 byte	0x0A 1 byte	0x00 1 byte	0x06 1 byte	0x7D 1 byte	0x84 1 byte

ADU (Application Data Unit)

2) ODPOVĚĎ: SPRÁVNÁ ODPOVĚĎ OD SLAVE

Adresa zařízení	Kód funkce	Počet datových bytů	Stav cívek (data)	CRC (CRC-16)	
			Byte 0	Low byte	High byte
0x01 1 byte	0x01 1 byte	0x01 1 byte	0x2D 1 byte	0x91 1 byte	0xB8 1 byte

ADU (Application Data Unit)

- Čtení 6 cívek od adresy 10,
- cívky mají stav $(101101)_2$ v pořadí **LSB**→**MSB**.

Příklad výjmkky

1) ZPRÁVA: ČTI CÍVKY (Read Coils) – požadavek od Master → Slave

Adresa zařízení	Kód funkce	Počáteční adresa		Počet cívek		CRC (CRC-16)	
		Hi byte	Lo byte	Hi byte	Lo byte	Low byte	High byte
0x01 1 byte	0x01 1 byte	0x00 1 byte	0x0A 1 byte	0x00 1 byte	0x06 1 byte	0x7D 1 byte	0x84 1 byte

ADU (Application Data Unit)

2) ODPOVĚĎ: CHYBOVÁ ODPOVĚĎ OD SLAVE – NEEXISTUJÍCÍ ROZSAH

Adresa zařízení	Kód funkce	Kód výjimky (Exception Code)	CRC (CRC-16)	
			Low byte	High byte
0x01 1 byte	0x81 1 byte (0x01 + 0x80)	0x02 1 byte	0xC0 1 byte	0xF1 1 byte

ADU (Application Data Unit)

- Kód funkce = požadovaný kód + 0x80,
- v datech je předán kód výjimky.

Běžné kódy výjimek

Kód	Význam
01	Illegal Function – zařízení nepodporuje požadovaný funkční kód
02	Illegal Data Address – požadovaná adresa nebo rozsah registrů/cívek neexistuje
03	Illegal Data Value – neplatná hodnota parametru v požadavku
04	Slave Device Failure – interní chyba zařízení při zpracování požadavku
05	Acknowledge – zařízení požadavek přijalo, ale zpracování trvá delší dobu
06	Slave Device Busy – zařízení je zaneprázdněno a nemůže požadavek zpracovat
08	Memory Parity Error – chyba paměti nebo kontrolního součtu v zařízení
10	Gateway Path Unavailable – gateway nemá dostupnou komunikační cestu
11	Gateway Target Device Failed to Respond – cílové zařízení neodpovědělo gateway

Výhody protokolu modbus

- Velmi jednoduchá programová implementace (lze naprogramovat od základu),
- otevřený protokol - množství knihoven,
- malá režie,
- přenos registrů:
 - teplota,
 - tlak,
 - výkon, ...
- přenos binárních stavů:
 - tlačítka,
 - relé,
 - alarmy, ...
- je dostatečně rychlý pro SCADA systém,
- zajišťuje kontrolu dat CRC.

Nevýhody protokolu modbus

- Nezná datový typ **string**:
 - do jednoho registru lze teoreticky ručně vložit dva ASCII znaky,
 - přes registry lze uložit jen malé množství textu.
- nemá funkci pro automatický popis dat (porovnejte s **OPC UA**, kde tato funkce je),
- není zde automatická možnost vyhledání připojených zařízení,
- klasický modbus nemá šifrování - i modbus TCP lze odposlouchávat například metodou **Man-in-the-Middle (MitM)**,
- nemá časové značky.

Modbus je jednoduchý a univerzální průmyslový protokol vhodný pro přenos registrů a binárních dat, ale nepodporuje komplexní datové typy, metadata ani zabezpečení komunikace.

Téma

- 1 Popis protokolu
- 2 Ukázka komunikace**
- 3 Tématické okruhy ke zkoušce

Zadání

- Vytvořit python skript zajišťující spuštění modbus TCP serveru (**pozor, toto je slave !!!**),
- server bude mít lokální přístup na IP adrese 127.0.0.1:502,
- server bude poskytovat následující data:
 - fyzická adresa 0 až 4 pro cívky, volně nastavitelný stav,
 - fyzická adresa 0 až 4 pro digitální vstupy, náhodně měnící se stav,
 - fyzická adresa 0 až 4 pro vstupní registry, náhodně měnící se hodnota,
 - fyzická adresa 0 až 4 pro uchovávací registry, volně nastavitelná hodnota,
- server bude otestován programem QModMaster.

Zadání

- Vytvořit python skript zajišťující spuštění modbus TCP serveru (**pozor, toto je slave !!!**),
- server bude mít lokální přístup na IP adrese 127.0.0.1:502,
- server bude poskytovat následující data:
 - fyzická adresa 0 až 4 pro cívky, volně nastavitelný stav,
 - fyzická adresa 0 až 4 pro digitální vstupy, náhodně měnící se stav,
 - fyzická adresa 0 až 4 pro vstupní registry, náhodně měnící se hodnota,
 - fyzická adresa 0 až 4 pro uchovávací registry, volně nastavitelná hodnota,
- server bude otestován programem QModMaster.

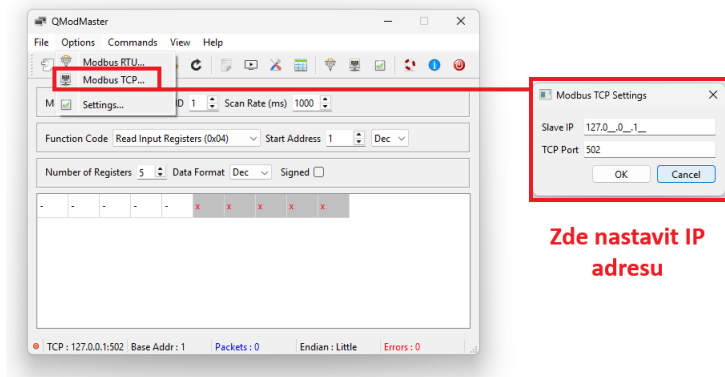
Python skript - data

```
11 import random
12 import threading
13 import time
14
15 from pymodbus.server import StartTcpServer
16 from pymodbus.datastore import (
17     ModbusSequentialDataBlock,
18     ModbusSlaveContext,
19     ModbusServerContext,
20 )
21
22 # Založení paměťového prostoru
23 store = ModbusSlaveContext(
24     co=ModbusSequentialDataBlock(0, [False, False, False, False, False]),
25     di=ModbusSequentialDataBlock(0, [True, True, False, False, True]),
26     hr=ModbusSequentialDataBlock(0, [100, 200, 300, 400, 500]),
27     ir=ModbusSequentialDataBlock(0, [10, 20, 30, 40, 50]),
28 )
29
30 # registrace obsahu
31 context = ModbusServerContext(slaves=store, single=True)
32
```

Python skript - vlákna

```
33 # vlákno náhodného nastavení vstupů
34 def update_inputs():
35     while True:
36         slave = context[0]
37
38         # Discrete Inputs, funkce 02
39         di = [random.choice([False, True]) for _ in range(5)]
40         slave.setValues(2, 0, di)
41
42         # Input Registers, funkce 04
43         ir = [random.randint(0, 1000) for _ in range(5)]
44         slave.setValues(4, 0, ir)
45
46         print("DI:", di, "IR:", ir)
47         time.sleep(4)
48
49
50 # start vlákna nastavování vstupů
51 threading.Thread(target=update_inputs, daemon=True).start()
52
53
54 # start vlákna modbus serveru
55 StartTcpServer(
56     context=context,
57     address=("127.0.0.1", 502),
58 )
```

QModMaster - nastavení IP adresy serveru



QModMaster - ovládání

Připojení k serveru

Provedení jednoho čtení nebo zápisu

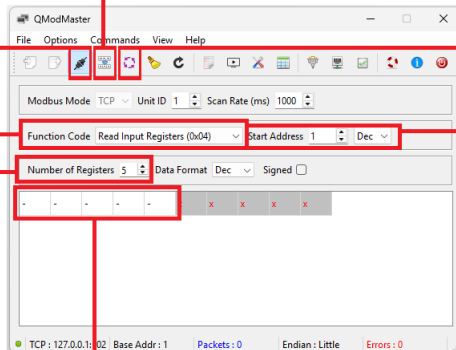
Provádění periodického čtení a zápisu

Typ operace

Počet dat

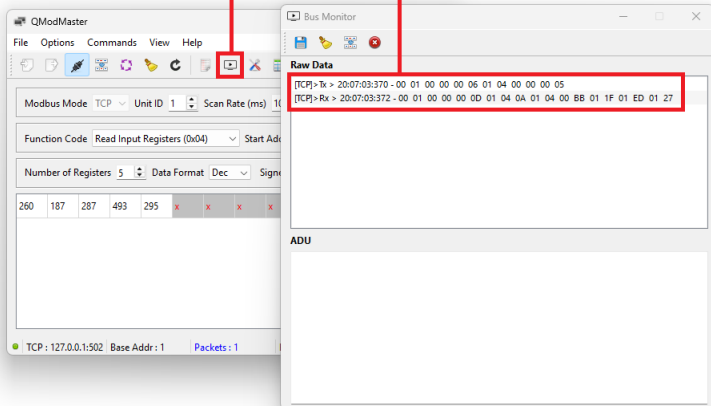
Nastavení počáteční
adresy dat
(logická adresa)

Přečtená nebo nastavená hodnota dat



QModMaster - monitor

Hexadecimální výpis zpráv



QModMaster - k vyzkoušení

- zápis nebo čtení od adresy 1 více jak 5 a méně jak 9 binárních dat,
- zápis nebo čtení od adresy 1 více jak 8 binárních dat,
- zápis nebo čtení od adresy 1 více jak 5 číselných dat.

Téma

- 1 Popis protokolu
- 2 Ukázka komunikace
- 3 Tématické okruhy ke zkoušce**

Číselné soustavy - okruh 10 bodů

- ❶ Poziční číselná soustava, celočíselná část, neceločíselná část (**přesnost!!!**).
- ❷ Převod do desítkové soustavy, přímé vyjádření a Hornerovo schéma.
- ❸ Převod do jiné než desítkové soustavy, postupné dělení (násobení) a hledání mocnin.
- ❹ Převody mezi soustavami, kde je základ jedné mocninou té druhé - proč to lze?
- ❺ Sčítání (odčítání s přímým vyjádřením znaménka), přetečení, délka operandů a výsledku.
- ❻ Násobení, příklad pro binární, délka operandů a výsledku.
- ❼ Přímý kód. Jak se provádějí aritmetické operace?
- ❽ Aditivní kód. Jak se provádějí aritmetické operace?
- ❾ Doplnkový kód. Jak se provádějí aritmetické operace?
- ❿ Pohyblivá řádová čárka IEEE-754, stačí jednoduchá přesnost, ale vědět i speciální hodnoty.

Logické obvody - okruh 10 bodů

- 1 Popis logické funkce, tabulka, rovnice, mapa.
- 2 Booleova algebra, zákony - vyzkoušíme na úpravě konkrétní rovnice.
- 3 Základní logické operace, AND, OR, NOT a XOR, liniové schéma.
- 4 De Morganovy zákony, co platí, ukázat na úpravě rovnice.
- 5 Booleovská krychle, Karnaughova mapa, tvorba, zápis stavů, minimalizace.
- 6 Kombinační obvody, dekodér, multiplexer, demultiplexer, sčítačka.
- 7 Sekvenční logická funkce, diagram, logický automat.
- 8 Klopné obvody RS, D, zapojení posuvného registru nebo čítače.
- 9 Hazardy.
- 10 Třístavový výstup a přístup na sběrnici.
- 11 ALU, co dělá, příznaky, akumulátor atd.

Mikroprocesory - okruh 10 bodů

- ❶ Paměť RAM, FLASH, EEPROM, reprezentace dat Big a Little Endian.
- ❷ Typy architektur, von Neumannova a Harvardská, rozdíly, problémy.
- ❸ Instrukční sada CISC a RISC.
- ❹ STM8 CPU registry, PC, SP, A, X, Y, CC.
- ❺ Instrukce a strojový kód, pipeline, instrukce skoků.
- ❻ Podmínková větvení v assembleru.
- ❼ Přímá adresace a nepřímá adresace.
- ❽ Indexová adresace a relativní adresace.
- ❾ Zásobníková paměť, k čemu slouží, přístup a adresace.
- ❿ Funkce a přerušení, rozdíly a práce se zásobníkem.
- ⓫ Periferie procesoru, konfigurace, stav po resetu apod.

Záchranná otázka - 4 body

- 1 BCD kód.
- 2 Parita.
- 3 Práce s maskou registru.
- 4 Rozdíl mezi asynchronním a synchronním čítačem.
- 5 SCADA, k čemu je to dobré.
- 6 OPC server, k čemu je to dobré.
- 7 Control Web, přístroje, proměnné, procedury, ovladače.
- 8 Rozdíl mezi synchronní a asynchronní komunikací.
- 9 UART komunikace.
- 10 SPI komunikace.

Poslední přednáška

Děkuji za pozornost